Hospital Management System – Project Report

1. Student Details

Name: Sravan MP
Roll Number: 21f3000245
Email: 21f3000245@ds.study.iitm.ac.in
About Me: I'm an Associate software engineer At oracle financial services limited completing my diploma from IIT MADRAS this year.

---

2. Project Details

Project Title: Hospital Management System

Problem Statement

To design and build a system that digitalizes hospital workflows—patient registration, doctor discovery, appointment booking, treatment tracking, and medical history management—to reduce errors and administrative overhead.

Approach

The system is built using Flask with a modular backend design. Role-based dashboards were implemented for Admins, Doctors, and Patients. Vue.js powers the responsive frontend. Redis caching and Celery background tasks were integrated to handle performance-heavy operations.

---

3. AI/LLM Declaration

I utilized GitHub Copilot and ChatGPT during the development process primarily for generating styled HTML email templates, refactoring Vue.js components for better structure, and improving my understanding of Redis caching best practices.

The overall extent of AI/LLM usage was approximately 20-25%. This support was mainly in the form of code suggestions, syntax corrections, formatting assistance for documentation, and helping me identify certain edge cases in the appointment booking workflow.

All core system design, business logic implementation, API architecture, database modeling, and debugging were performed manually.

4. Technologies and Frameworks Used

Technology / Library	Purpose
Flask	Backend web framework
SQLAlchemy	ORM for database modelling
Vue.js 3	Frontend SPA framework
Redis	Caching layer
Celery	Background task processing
Chart.js	Data visualization
Bootstrap 5	Responsive UI styling
JWT	Token-based authentication
SQLite	Development database

---

5. Database Design

Core Tables:

- **User** – Credentials, role, and profile data
- **DoctorProfile** – Specialization, department, availability
- **PatientProfile** – Medical details and contact info
- **Department** – Hospital departments
- **Appointment** – Booking data (with unique (doctor, date, time) constraint)
- **DoctorAvailability** – Time slots and fees
- **Treatment** – Diagnosis and prescriptions

Key Relationships:

- User → DoctorProfile/PatientProfile (One-to-One)
- Department → DoctorProfile (One-to-Many)
- DoctorProfile → Appointment (One-to-Many)
- Appointment → Treatment (One-to-One)

---

6. API Endpoints Summary

Authentication

`API Endpoints DocumentationAuthentication Endpoints`

- `POST /api/auth`
- `PUT /api/auth`

Patient EndpointsDoctors & Availability

- `GET /api/doctors/search`: Find and search for doctors.
- `GET /api/doctors/<id>/slots`: Retrieve the time slots available for a specific doctor.

Appointments

- `POST /api/appointments`: Create a new appointment record.
- `PATCH /api/appointments/<id>`: Modify an existing appointment.
- `DELETE /api/appointments/<id>`: Cancel or remove an appointment.

Patient Operations

- `GET /api/patient/dashboard`: Access and retrieve the patient's dashboard information.
- `POST /api/patient/export/async`: Request an asynchronous export of the patient's data.

Doctor Endpoints

- `POST /api/doctor/availability`: Set or update the doctor's availability.
- `GET /api/doctor/dashboard`: Access the doctor's dashboard.
- `POST /api/appointments/<id>/treatment`: Record the treatment details for a specific appointment.
- `GET /api/doctor/patients/<id>`: Retrieve information for a specific patient of the doctor.

Admin Endpoints

- `GET/POST /api/admin/departments`: Retrieve or create hospital departments.
- `GET/POST /api/admin/doctors`: Retrieve or create doctor profiles.
- `PUT /api/admin/doctors/<id>`: Update an existing doctor's profile.
- `GET /api/admin/analytics`: Access administrative analytics and reporting data.

---

7. Features & Implementation

Appointment System

Generates dynamic slots based on doctor availability. Double booking is prevented using database-level unique constraints.

Caching Strategy

Redis caches doctor search and appointment dashboards. Cache invalidation uses scan-based key removal.

Background Jobs

Celery handles: daily reminders, monthly reports, and CSV data exports.

Role-Based Dashboards

Admins, Doctors, and Patients have individual dashboards with real-time data visualization through Chart.js.

---

8. Challenges & Solutions

Double Booking: Solved using unique DB index and error handling.
Stale Cache: Added cache invalidation logic and synchronized user/doctor status fields.
Chart Rendering Issues: Fixed using Vue's nextTick().
Celery on Windows: Used `--pool=solo`.

---

9. Learning Outcomes

- Database normalization & concurrency handling
- Practical caching and performance optimization
- JWT security implementation
- Real-world debugging experience

---

10. Conclusion

The project demonstrates a full-stack Hospital Management System with production-grade practices—authentication, caching, background tasks, dashboards, and scalable design.

8. Video Presentation

Drive Link:
21f3000245.mp4